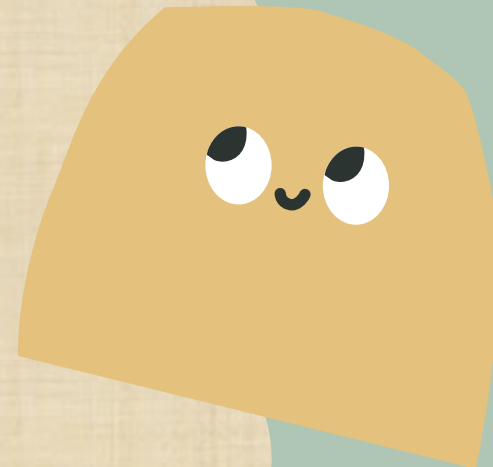
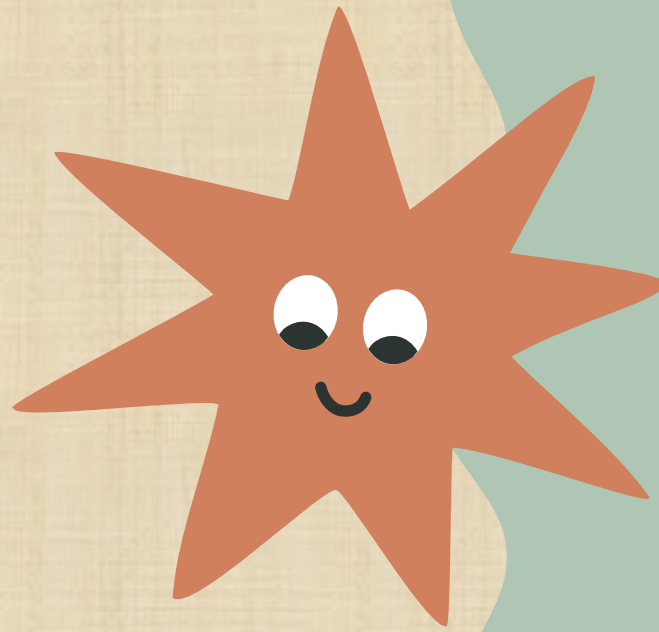


UNIT 3

INPUT/OUTPUT HANDLING IN C++

10TH COMPUTER SCIENCE
SINDH TEXTBOOK BOARD

<https://awinashgoswami.github.io>



Agenda

- Basic Structure of C++
- Comment Statement
- Output Function
- Input Function
- Statement Terminator
- Escape Sequence
- Operators

Basic Structure in C++

Code	Explanation
<code>#include<iostream></code>	Header File
<code>using namespace std;</code>	Namespace declaration
<code>int main()</code>	Execution begins at Main
<code>{</code>	Opening Bracket
<code>cout<<"Hello World";</code>	Print Statement
<code>}</code>	Closing Bracket

Comment Statement

- The comment statement are those statement which are ignored by compiler.
- They are used for describing code.
- There are two types of comment statements. Single Line and Multi-line comments
 - Single Line Comment: It covers single line and start from double slash (//). E.g. //this is variable
 - Multiple Line Comment: It covers multiple lines. The text of the comment is placed inside /* */
e.g. /*This is multiple line comment*/

Output Handling

Functions	Description	Example
cout	Cout is a predefined object in C++. It is used to display the output device i.e. monitor. "cout" uses insertion operator (<<)	<pre>#include<iostream> using namespace std; int main() { cout<<"Hello World"; }</pre>
puts()	The function is used to print the string to the output stream.	<pre>#include<iostream> using namespace std; int main() { puts("Hello World"); }</pre>

Input Handling

Functions	Description	Example
cin	cin is an object that reads data from keyboard with extraction operator (>>).	<pre>#include<iostream> using namespace std; int main() { int a; cin>>a; }</pre>
getch()	The function is defined in conio.h (console input/output header file) and used to get a single character from keyboard.	<pre>#include<iostream> #include<conio.h> using namespace std; int main() { char a = getch(); cout<<a; }</pre>

Input Handling Cont....

Functions	Description	Example
gets()	The function is defined in stdio.h (console input/output header file) and used to get characters string from keyboard.	<pre>#include<iostream> #include<stdio.h> using namespace std; int main() { char a[20]; gets(a); cout<<a; }</pre>

Input Handling Cont....

Functions	Description	Example
getche()	The function is defined in conio.h (console input/output header file) and used to get a single character from keyboard and also display the character.	<pre>#include<iostream> #include<conio.h> using namespace std; int main() { char a = getche(); cout<<a; }</pre>
getchar()	The function is defined in stdio.h (console input/output header file) and used to get a single character from keyboard and need to press enter.	<pre>#include<iostream> #include<stdio.h> using namespace std; int main() { char a = getchar(); cout<<a; }</pre>

Statement Terminator

Every statement in C++ must be terminated with a semicolon (;). It indicates the end of the statement and is also called the Statement terminator. If the terminator is missing, an error message will occur.

Escape Sequences

- Escape sequences are special non-printing characters. They can be used with cout in C++. An escape sequence starts with a backslash (\) and a code character.

Escape Sequence	Explanation with Example
\a	"a" means Alert or alarm. It causes a beep sound in the computer. Example: cout << "\a";
\t	"t" stands for Horizontal tab. It is used to shift the cursor to a couple of spaces to the right in the same line. Example: cout << "\t";
\n	"n" stands for New line or line feed. It inserts a new line and cursor moves to the beginning of the next line. Example: cout << "\n";
\\	Backslash. It is used to print the backslash character. Example: cout << "\\";
\'	Single quotation. It is used to print the "apostrophe (')" sign or character. Example: cout << "\'";
\"	Double quotation. It is used to print the quotation mark. Example: cout << "\"";

Operators

- Operators are special symbols used for specific purposes. Operators perform mathematical operations on Operands. For example: $x + y$. Here "x" and "y" are operands and "+" is operator. There are seven types of operators used in C++ programming:
 - Arithmetic Operators
 - Increment Operators
 - Decrement Operators
 - Relational Operators
 - Logical Operators
 - Assignment Operators
 - Arithmetic Assignment Operators

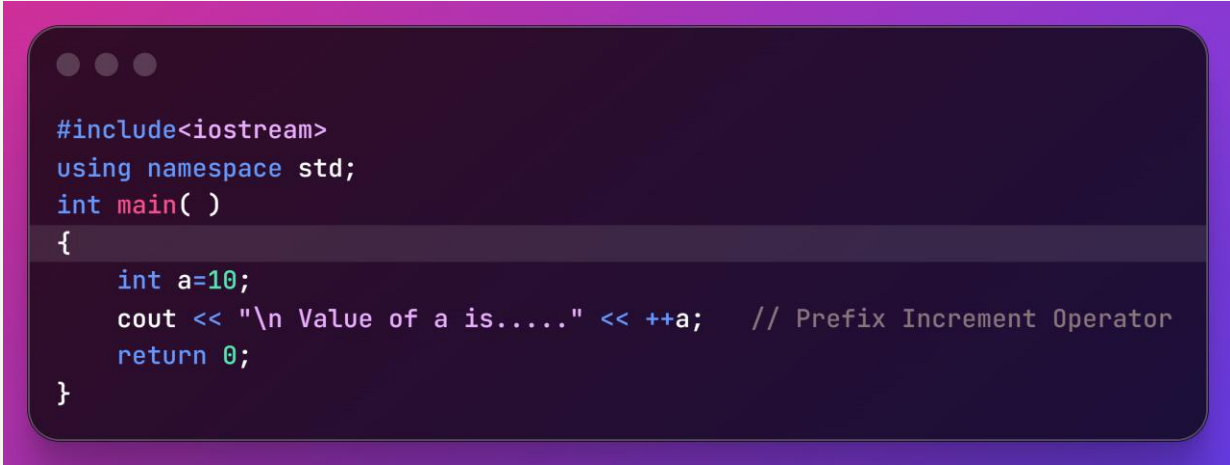
Arithmetic Operator

- In Arithmetic operators, five different operators are used to perform an arithmetic operation.
- **Addition (+)**: It is used to perform arithmetic addition. **Example:** $a + b$;
- **Subtraction (-)**: It is used to perform arithmetic subtraction. **Example:** $a - b$;
- **Multiplication (*)**: It is used to perform arithmetic multiplication. **Example:** $a * b$;
- **Division (/)**: It performs the arithmetic division of two numbers. **Example:** a / b ;
- **Remainder / Modulus (%)**: It is used to find remainder of a division. It returns the remainder of an integer value. Remainder operator is also known as Modulus operator. This operator is used only with integral data types. **Example:** $5 / 2 = 2$ and 1 is remainder. We can write in this form $5 \% 2$

```
#include<iostream>
using namespace std;
int main( )
{
    int a,b,add,sub,multi,remd;
    float div;
    cout <<"\n \t CALCULATOR";
    cout <<"\n \t =====";
    cout << "\n \t Enter the Value of a.....";
    cin >> a;
    cout << "\n \t Enter the value of b.....";
    cin >> b;
    add = a+b;
    cout << "\n \t Addition of "<< a <<"and.. "<< b <<" is...."<< add;
    sub=a-b;
    cout << "\n \t Subtraction of "<< a <<"and..."<< b <<" is...."<< sub;
    multi=a*b;
    cout << "\n \t Multiplication of "<< a <<"and..."<< b <<" is...."<< multi;
    div=a/b;
    cout << "\n \t Division of "<< a <<"and..."<< b <<" is...."<< div;
    remd=a%b;
    cout << "\n \t Remainder of Modulus division of "<< a <<"and..."<< b <<" is...."<< remd;
    return 0;
}
```

Increment Operator

- The increment operator can be used with any type of variable. It is used to add 1 to the value of a variable. Increment operator represented by ++ (double plus sign). This operator can be applied only to a single variable. There are two ways to use increment operator:
- **Prefix Increment Operator:** You can apply this operator before the variable name. It can be written like ++a, i.e. x = ++a;



```
#include<iostream>
using namespace std;
int main( )
{
    int a=10;
    cout << "\n Value of a is....." << ++a;    // Prefix Increment Operator
    return 0;
}
```

- **Explanation:** In prefix increment operation value of a is printed as 11 because 1 is added in a before printing.

Increment Operator Cont....

- **Postfix Increment Operator:** If the increment operator is applied **after** the variable name, it is known as **Postfix Increment operator**. It is written like a++.



```
#include<iostream>
using namespace std;
int main( )
{
    int a=10;
    cout << "\n Value of a is....." << a++;    // Postfix Increment Operator
    return 0;
}
```

- **Explanation:** In postfix increment operation, value of a is printed as 10 because 1 is added in a **after** printing. So the value of a will change to 11 after printing.

Decrement Operator

- The decrement operator is same as increment operator but it subtracts 1 from the value of a variable. It is represented by -- (double minus sign). It can also be used as prefix and postfix.

```
#include<iostream>
using namespace std;
int main( )
{
    int a=10;
    cout << "\n Value of a is....." << a--;
}
```


Relational Operators

- The relational operators are used to test the relation between two values. All relational operators are binary operators. These operators must require two operands. The result of the comparison is **True (1)** or **False (0)**. The relational operators are also known as **Comparison Operators**.

Operator	Meaning	Purpose	Expression
==	Equal to	It checks the equality of two operand values.	a == b
!=	Not equal to	It checks whether the value of the left operand is not equal to the value of the right operand.	a != b
>	Greater than	It checks whether the value of the left operand is greater than the value of the right operand.	a > b
<	Less than	It checks whether the value of the left operand is less than the value of the right operand.	a < b
>=	Greater than or equal to	It checks whether the value of the left operand is greater than or equal to the value of the right operand.	a >= b
<=	Less than or equal to	It checks whether the value of the left operand is less than or equal to the value of the right operand.	a <= b

```
// Relational Operators
#include <iostream>
using namespace std;

int main()
{
    int a = 10;
    int b = 20;

    cout << "\n\tRelational Operator";
    cout << "\n\t===== ";

    cout << "\n\t" << (a == b) << endl;
    cout << "\n\t" << (a < b) << endl;
    cout << "\n\t" << (a > b) << endl;
    cout << "\n\t" << (b <= a) << endl;
    cout << "\n\t" << (a >= a) << endl;
    cout << "\n\t" << (b != a) << endl;

    return 0;
}
```